06

Part 6 · Security

Identity, permissions and encryption: protecting data and access in the cloud.

- IAM
- Authentication vs Authorization
- Least Privilege
- Encryption at rest vs in transit
- Secrets, keys & compliance

6.1 IAM (Identity and Access Management)

★★★

IN ONE LINE

The system that defines who (or what) can perform which actions on which cloud resources.

IAM (Identity and Access Management) controls **who can do what on which resources** in the cloud. Every API call is checked against IAM, so it is the foundation of cloud security.

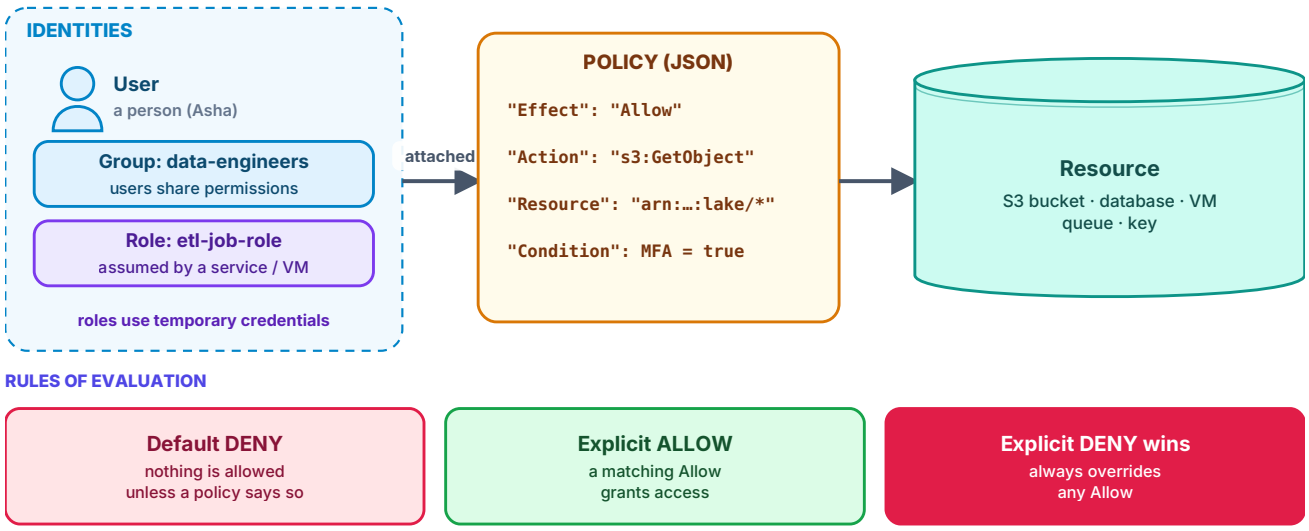


Figure 6.1 — Identities, policies and resources

Concept	Meaning
User	A human (or legacy app) with long-term credentials: password + MFA, access keys.
Group	A set of users that share the same policies (e.g. *data-engineers*).
Role	An identity with **temporary** credentials that a user, VM, function or another account *assumes*. Preferred for services.
Policy	A document that lists allowed or denied actions on resources, optionally with conditions.
Root / owner account	Full power. Protect with MFA and almost never use it.

AWS IAM POLICY · READ-ONLY ACCESS TO ONE PREFIX

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::my-lake", "arn:aws:s3:::my-lake/silver/*"]
  }]
}
```

INTERVIEW ANGLE

Services should never use access keys hard-coded in code. Give the VM / function / pipeline an **IAM role** and let the platform supply short-lived credentials. Mapping: AWS IAM → Azure Entra ID + RBAC → Google Cloud IAM.

6.2 Authentication vs Authorization

★★★

IN ONE LINE

Authentication proves identity; authorization decides what that identity may do.

Authentication (AuthN) proves **who you are**. **Authorization (AuthZ)** decides **what you are allowed to do**. You must authenticate first; then each request is authorized.

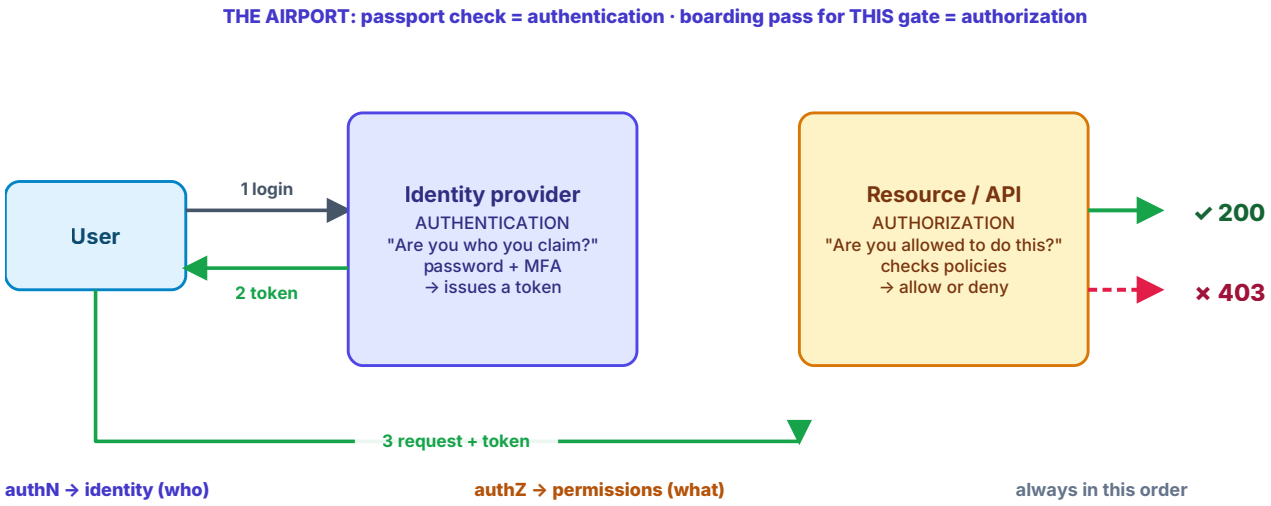


Figure 6.2 — Login, token, permission check

	Authentication	Authorization
Question	Who are you?	What may you do?
Methods	Password, MFA, biometrics, certificates, tokens, SSO	Policies, roles, ACLs, RBAC, scopes
Failure code	**401 Unauthorized** (not authenticated)	**403 Forbidden** (authenticated, not allowed)
Cloud example	Sign in to the console with MFA	IAM policy lets the role read one bucket

Multi-factor authentication

Combine **something you know** (password), **something you have** (phone / hardware key) and **something you are** (fingerprint). Blocks most stolen-password attacks.

Words you will hear

- **SSO** — one login for many apps.
- **SAML / OIDC** — standards for federated login (OIDC adds identity on top of **OAuth 2.0**).
- **OAuth 2.0** — delegated *authorization* ("let this app read my calendar").
- **JWT** — a signed token carrying claims.

6.3 Principle of Least Privilege

★★★

IN ONE LINE

Grant only the minimum permissions needed, for the minimum time.

The **Principle of Least Privilege (PoLP)**: give every user, role and service **only the minimum permissions needed, for only as long as needed**. It limits the damage (blast radius) from mistakes, bugs and compromised credentials.

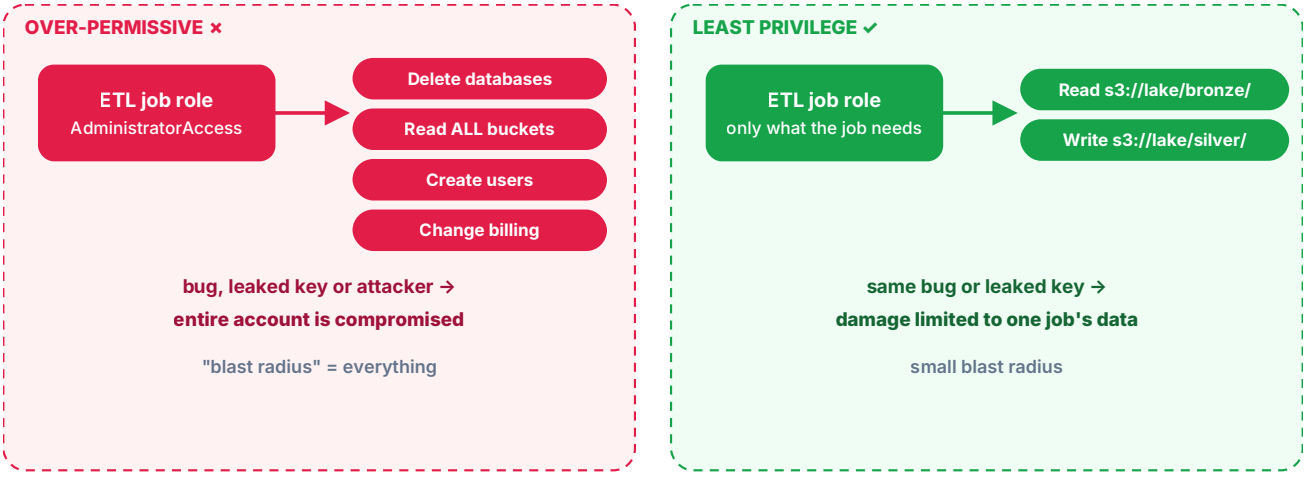


Figure 6.3 — Same failure, very different impact

How to apply it

- Start with **no access** and add what is required.
- Prefer **roles + groups** over individual grants.
- Scope by **resource** (one bucket / prefix), not `*`.
- Use **temporary credentials** and just-in-time elevation.
- Add **conditions** (MFA, source IP, time).
- Separate **dev / test / prod** accounts.

Keeping it that way

- Regularly review and remove unused permissions (access analyzers).
- Rotate or delete old keys.
- Log and alert on privileged actions.
- Avoid wildcards like `Action: "*"` and `Resource: "*"`.

INTERVIEW ANGLE

Related ideas: **separation of duties** (no one person can do everything), **defence in depth** (many independent layers: network + IAM + encryption + monitoring) and **zero trust** (“never trust, always verify”, even inside the network).

6.4 Encryption at Rest vs Encryption in Transit

★★★

IN ONE LINE

Protect stored data with keys on disk; protect moving data with TLS.

Encryption turns readable data into unreadable ciphertext that only holders of the right **key** can decode. Protect data in both of its states: **at rest** (stored) and **in transit** (moving). A third state, **in use**, is covered by confidential computing.

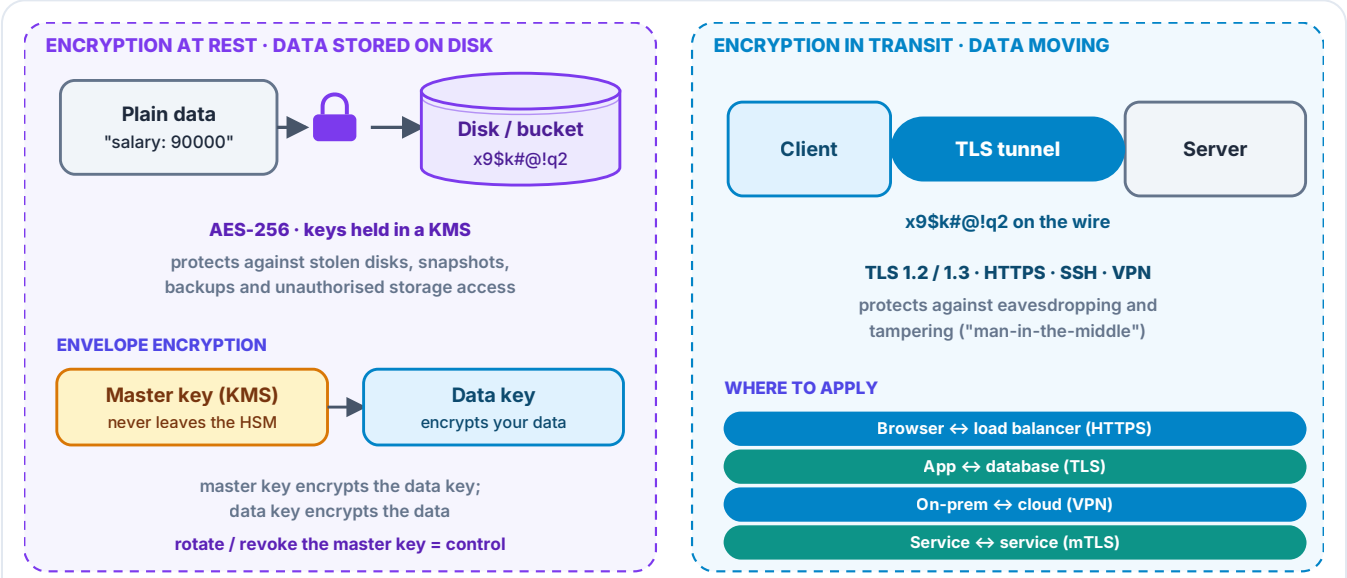


Figure 6.4 — Two states of data, two protections

	At rest	In transit
Protects	Stored data: disks, buckets, databases, backups	Data moving across a network
Threat	Stolen disk / snapshot, insider, storage breach	Eavesdropping, tampering, man-in-the-middle
Typical tech	AES-256, server-side encryption, KMS-managed keys, TDE for databases	TLS 1.2/1.3 (HTTPS), SSH, IPsec VPN
Cloud default	Often on by default for object storage and disks	You must enforce HTTPS / TLS
Key question	Who controls the key?	Is the certificate valid and trusted?

Key management choices

- **Provider-managed** keys — easiest.
- **Customer-managed** keys (CMK) in KMS — you control rotation and access.
- **Bring/hold your own key** (BYOK/HYOK) — strict compliance.

Symmetric vs asymmetric

Symmetric (AES): one shared key, fast, for bulk data.
Asymmetric (RSA, ECC): public/private key pair, used for key exchange and signatures, e.g. in TLS handshakes.

REMEMBER

Encrypt **at rest** so a stolen disk is useless; encrypt **in transit** so network spies see nothing. Encryption does not replace access control: a user with permission to read the decrypted data still can.

6.5 Secrets, Key Management & Compliance

★★☆

IN ONE LINE

Protecting credentials and keys, defending in layers, and meeting legal and audit requirements.

Beyond IAM and encryption, a few more security concepts regularly come up in fresher cloud interviews.

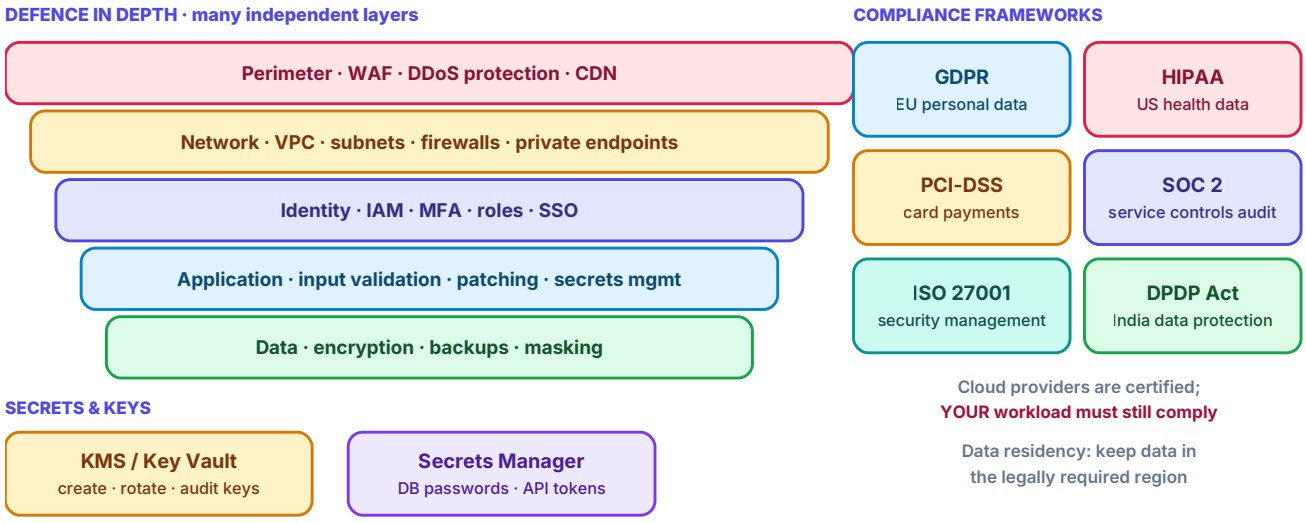


Figure 6.5 — Layers of defence, secrets management, and compliance

Concept	Why it matters
Secrets management	Passwords, API keys and tokens belong in a vault (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) — **never in code, Git or images**. Rotate them automatically.
KMS (key management service)	Creates and protects encryption keys in hardware security modules (HSMs); logs every use.
DDoS protection	Absorbs floods of malicious traffic (AWS Shield, Azure DDoS Protection, Cloud Armor).
Security monitoring	Audit logs (CloudTrail, Azure Activity Log) + threat detection (GuardDuty, Defender, Security Command Center).
Compliance & residency	Laws and standards dictate where data may live and how it must be protected; providers supply audit reports, you configure correctly.
Common attacks	Leaked keys, public buckets, weak passwords / no MFA, unpatched VMs, SQL injection, phishing.

COMMON TRAP

The most common real incident: **an access key accidentally committed to a public Git repository**. Bots find it in minutes. Use roles, secret managers and pre-commit secret scanners; if a key leaks, **revoke it immediately** and rotate.

DATA ENGINEERING TIE-IN

Mask or tokenise PII on its way from Bronze to Silver, restrict access with table/column permissions (e.g. Unity Catalog), and keep data in the allowed region. Credentials for pipelines come from a secret store, not config files.